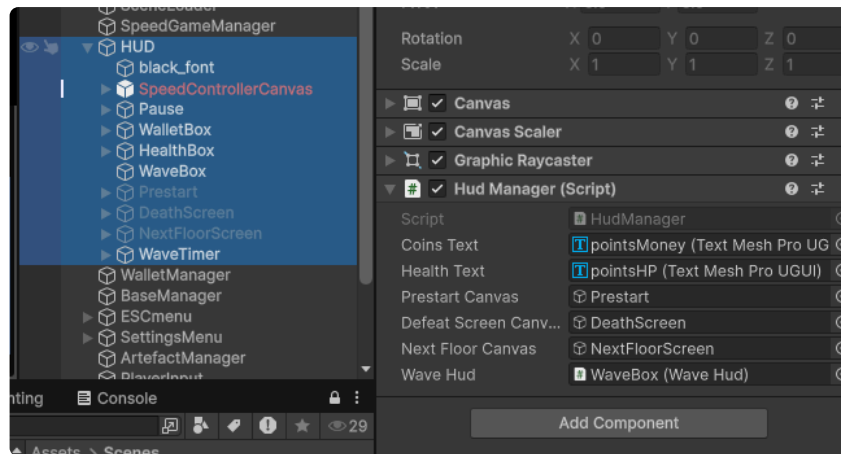


Чтобы некоторые элементы интерфейса чувствовали себя более комфортно и работать с ними было тоже куда более проще - они были собраны в один единый HUD и вынесены в Persistent сцену. HUD просто включается и выключается когда нужно (его вызывает GameFlowManager).



Верхняя панель интерфейса.



Что входит в HUD. Hud Manager.

Я также сделал дочерними PrestartScreen, DeathScreen, NextFloorScreen. Потому что это позволит мне более деликатно ими управлять и более того, они появляются только при включенном HUD. Тот же SettingsController не является дочерним, потому что может быть вызван как в игре, когда HUD включен, так и из Main_Menu, когда HUD выключен.

```
1 using TMPro;
2 using UnityEngine;
3 using UnityEngine;
4 using UnityEngine.InputSystem;
5
6 public class HudManager : Manager<HudManager>
7 {
8     [SerializeField] private TMP_Text coinsText;
9     [SerializeField] private TMP_Text healthText;
10
11     [SerializeField] private GameObject prestartCanvas;
12     [SerializeField] private GameObject defeatScreenCanvas;
13     [SerializeField] private GameObject nextFloorCanvas;
14
15     [SerializeField] private GameObject timerWaveCanvas;
16     [SerializeField] private TextMeshProUGUI timerTXT;
17
18     [SerializeField] private WaveHud waveHud;
19 }
```

```

20     private PrestartUI prestartUI;
21     private DefeatScreen defeatScreen;
22     private NextFloorScreen nextFloorScreen;
23
24     public bool IsHudOn { get; private set; } = false;
25
26     protected override void Awake()
27     {
28         base.Awake();
29
30         defeatScreen = defeatScreenCanvas.GetComponent<DefeatScreen>();
31         prestartUI = prestartCanvas.GetComponent<PrestartUI>();
32         nextFloorScreen = nextFloorCanvas.GetComponent<NextFloorScreen>();
33         if (waveHud == null) { waveHud = GetComponent<WaveHud>(); }
34
35         HideDefeatScreen();
36     }
37
38     // В Awake() я получаю все нужные скрипты с дочерних компонентов:
39
40     /*
41     После чего в Start() я вызываю Reset(), делаю подписки и скрываю сам
42     Правильнее было бы держать скрипт родительским объектом, а главный кан
43     как дочерний и выключать его, чтобы менеджер всегда жил. Моё текущее р
44     может вызвать в редких случаях некоторые проблемы. Поэтому возможно я
45     дальнейшем изменю его на более правильный вариант.
46     */
47
48     private void Start()
49     {
50         Reset();
51
52         BaseManager.Instance.OnHealthChanged += UpdateHealthUI;
53         WalletManager.Instance.OnCoinsChanged += UpdateCoinsUI;
54
55         prestartUI.OnPressed += CallStartPlaying;
56
57         Hide();
58     }
59
60     // Методы обновления UI принимают данные и отображают их на экране.
61
62     private void UpdateHealthUI(float x, float health)
63     {
64         healthText.text = health.ToString();
65     }
66
67     private void UpdateCoinsUI(int coins)
68     {
69         coinsText.text = coins.ToString();
70     }
71
72     // Show() - включает объект и обновляет поля (на случай, если данные о
73
74     public void Show()
75     {
76         IsHudOn = true;
77         this.gameObject.SetActive(true);
78         UpdateInfoFields();
79     }
80
81     public void Hide()
82     {
83         IsHudOn = false;
84         this.gameObject.SetActive(false);
85     }
86
87
88     // Можно показать или скрыть окно пре-старта, окно экрана смерти, окно
89
90     public void HidePrestart()
91     {
92         prestartUI.Hide();
93         prestartCanvas.SetActive(false);
94     }
95
96     public void ShowPrestart()
97     {
98         prestartCanvas.SetActive(true);
99     }
100
101     public void ShowPrestartButton()

```

```

102     {
103         prestartUI.Show();
104     }
105
106     public void ShowDefeatScreen()
107     {
108         defeatScreen.ShowStatsRun();
109         defeatScreenCanvas.SetActive(true);
110     }
111
112     public void HideDefeatScreen()
113     {
114         defeatScreenCanvas.SetActive(false);
115     }
116
117     public void ShowStageCompletedCanvas()
118     {
119         nextFloorScreen.Show();
120     }
121
122     private void OnDestroy()
123     {
124         if (BaseManager.Instance != null)
125             BaseManager.Instance.OnHealthChanged -= UpdateHealthUI;
126
127         if (WalletManager.Instance != null)
128             WalletManager.Instance.OnCoinsChanged -= UpdateCoinsUI;
129
130         //prestartUI.OnPressed -= CallStartPlaying;
131     }
132
133     // На случай, если объект всё же будет уничтожен - OnDestroy (хотя объ
134     // является синглтоном и не должен уничтожаться в течении всего цикла
135
136     private void CallStartPlaying()
137     {
138         GameFlowManager.Instance.StartPlaying();
139     }
140
141     private void UpdateInfoFields()
142     {
143         coinsText.text = WalletManager.Instance.Coins.ToString();
144         healthText.text = BaseManager.Instance.MaxHp.ToString();
145     }
146
147     // UpdateFields - подтягивает информацию из менеджеров.
148
149     public void Reset()
150     {
151         HideDefeatScreen();
152         nextFloorScreen.Hide();
153         ShowPrestart();
154         UpdateInfoFields();
155     }
156
157     public void SetTotalWaveIcons(int totalwave)
158     {
159         waveHud.Init(totalwave);
160     }
161
162     public void SetCurrentWaveIcon(int wave)
163     {
164         waveHud.SetCurrentWave(wave);
165     }
166
167     // Для работы с отображением волн ещё два метода - установить максимум
168     // установить текущую волну.
169
170     public void TimerWaveHide()
171     {
172         if (timerWaveCanvas != null)
173             timerWaveCanvas.SetActive(false);
174     }
175
176     public void TimerWaveShow()
177     {
178         if (timerWaveCanvas != null)
179             timerWaveCanvas.SetActive(true);
180     }
181
182     public void TimerWaveSet(float time)
183     {
184         if (timerTXT == null) return;
185
186         timerTXT.text =

```

```

184         System.TimeSpan.FromSeconds(time)
185         .ToString(@"mm\:ss");
186     }
187 }

```

SpeedGameController.

Контролер скорости игры. Более подробно о нём расписано на странице SpeedGameManager. Дублировать сюда всю логику мне не хочется. Вкратце: есть менеджер скорости, который хранит значение множителя. А есть кнопки на интерфейсе - которые меняют это значение. Условный враг при высчитывании скорости движения умножает свою скорость на множитель скорости игры. Никаких Time.timeScale.

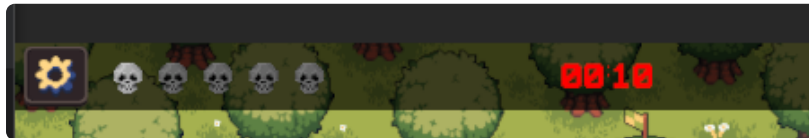
Управление доступно нажатием кнопок и клавишами: 1, 2, 3, 4.



SpeedGameController.

WaveBox.

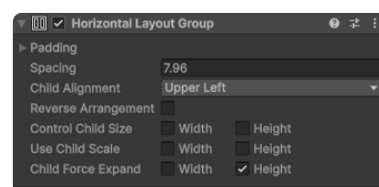
Для отображения всех волн и текущей волны, я сделал область с черепками, где подсвеченные - указывают прошедшее количество, а затемненные - сколько ещё осталось.



WaveBox.

На канвас WaveBox я повесил компонент для группировки элементов. Установил группировку по высоте и сделал 7.96 дистанцию. Теперь под этот канвас можно добавлять любые элементы и они все будут выравниваться в линию.

Я просто буду добавлять sprite черепков, а после пересобирать их. Если волн оказывается 3, то первые 3 будут естественными спрайтами, а оставшиеся - чуть прозрачнее.



Horizontal Layout Group.

```

1 using System.Collections.Generic;
2 using UnityEngine;
3 using UnityEngine.UI;
4
5 public class WaveHud : MonoBehaviour
6 {

```

```

7      [SerializeField] private Transform container;
8      [SerializeField] private Image prefabIcon;
9
10     [SerializeField] private Color activeColor = Color.white;
11     [SerializeField] private Color inactiveColor = new Color(1f, 1f, 1f);
12
13
14     private List<Image> icons = new();
15
16
17     public void Init(int totalWaves)
18     {
19         Clear();
20
21         for(int i = 0; i < totalWaves; i++)
22         {
23             var icon = Instantiate(prefabIcon, container);
24             icon.color = inactiveColor;
25             icons.Add(icon);
26         }
27     }
28
29     // Init() - Инициализируем количество волн и заполняем неактивными спрайтами
30     // всю панель. После в SetCurrentWave(int) получаем индекс текущей волны
31     // пересобираем линию спрайтов.
32
33     public void SetCurrentWave(int waveIndex)
34     {
35         for (int i = 0; i < icons.Count; i++)
36         {
37             icons[i].color = i < waveIndex
38                 ? activeColor
39                 : inactiveColor;
40         }
41     }
42
43     private void Clear()
44     {
45         foreach (var icon in icons)
46         {
47             Destroy(icon.gameObject);
48         }
49         icons.Clear();
50     }
51 }

```

Рядом на скрине можно заметить красный таймер. Учитывая, что в моей демо не планируется более 5-ти волн, то количество черепков никогда не достигнет его. Однако возможно его стоит перенести в другое место.

PrestartScreen.

Не самая обязательная штука, но было решено всё же её добавить. Как только затемнение экрана (переход FadeManager) спадает, то появляется текст: PRESS TO START. Это прозрачная кнопка, растянутая на весь экран, с лёгким затемнением.



Press to Start Button - PrestartScreen.

Нажать на кнопку можно мышкой, а можно - нажатием почти любой клавиши. Я сделал отдельную карту событий на отслеживание доступных клавиш, и добавил обработку такого события в этой класс: public void OnAnyKey(InputAction.CallbackContext ctx) - мы получаем калбек при нажатии. Если нажата любая клавиша → вызывается HandlePress. Он же вызывается при клике мышкой.

```
1 using System;
2 using TMPro;
3 using UnityEngine;
4 using UnityEngine.InputSystem;
5 using UnityEngine.UI;
6
7 public class PrestartUI : MonoBehaviour
8 {
9     [Header("UI Ref")]
10    [SerializeField] private Button prestartButton;
11    [SerializeField] private GameObject prestartText;
12
13    public event Action OnPressed;
14    private bool inputEnabled = false;
15
16    private void Awake()
17    {
18        prestartButton.onClick.AddListener(HandlePress);
19        Hide();
20    }
21
22    public void Show()
23    {
24        prestartText.SetActive(true);
25        prestartButton.interactable = true;
26        inputEnabled = true;
27    }
28
29    public void Hide()
30    {
31        prestartText.SetActive(false);
32        prestartButton.interactable = false;
33        inputEnabled = false;
34    }
35
36    public void HandlePress()
37    {
38        if (!inputEnabled) return;
39        inputEnabled = false;
40        prestartButton.interactable = false;
41
42        OnPressed?.Invoke();
43    }
44
45    /*
46     HandlePress() - делает кнопку не кликабельной, потому что ранее это в
47     - иногда получалось закликать кнопку, чтобы вызвать дублирование логики
48     Можно было не использовать событие, а просто вызывать GameFlowManager,
49     решил передать это действие в HudManager, который подписан на событие и
50     вызывает GameFlowManager.Instance.StartPlaying();
51     */
```

```

52
53     public void OnAnyKey(InputAction.CallbackContext ctx)
54     {
55         if (!ctx.performed) return;
56         HandlePress();
57     }
58 }

```

DeathScreen.

❗ Экран гибели называть таковым странно, как и называть его экраном поражения, ведь в бесконечной игре просто нельзя победить. Поэтому нас встречает вывеска THE END. А также следующая статистика: очки, убийства, волны, этажи, общее время забега (не ускоренное менеджером скорости).

У игрока есть кнопка Restart, чтобы начать сначала или кнопка выхода.



The End Screen.

Мы просто подгружаем на экран статистику из GameManager'a. А также предоставляем два метода под кнопки. Ультра-простенько...

К слову... Кнопки анимированны примерно также, как и в главном меню. Подробнее на странице Main Menu.

```

1  using TMPPro;
2  using UnityEngine;
3
4  public class DefeatScreen : MonoBehaviour
5  {
6      [SerializeField] private TMP_Text scoreTxt;
7      [SerializeField] private TMP_Text killsTxt;
8      [SerializeField] private TMP_Text wavesTxt;
9      [SerializeField] private TMP_Text floorTxt;
10     [SerializeField] private TMP_Text timeTxt;
11
12     private void Awake()
13     {
14         gameObject.SetActive(false);
15     }
16
17     public void ShowStatsRun() => ShowStats(GameManager.Instance.GetRun
18
19     private void ShowStats(RunStats stats)
20     {
21         scoreTxt.text = stats.score.ToString();
22         killsTxt.text = stats.kills.ToString();
23         wavesTxt.text = stats.waves.ToString();
24         floorTxt.text = stats.floor.ToString();
25         timeTxt.text = TimeFormatter.ToTime(stats.runTime);
26     }
27
28     public void OnRestart() => GameFlowManager.Instance.RestartGame();
29     public void OnLeave() => GameFlowManager.Instance.LeaveGame();

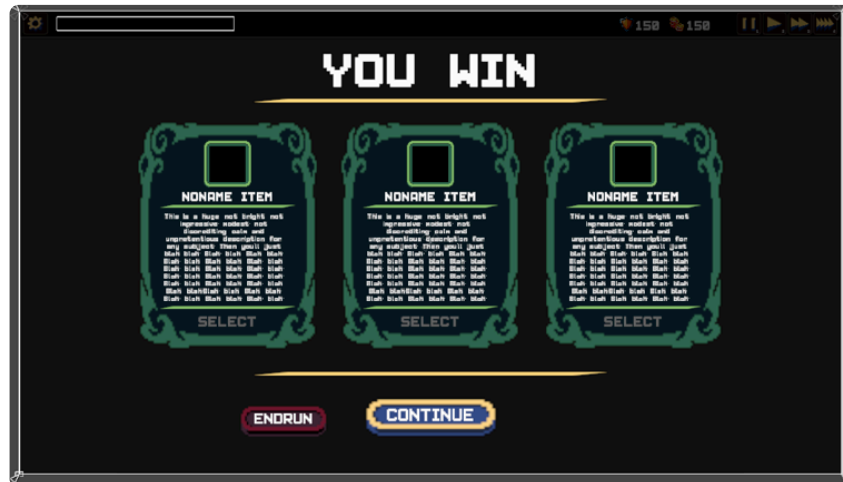
```

NextFloorScreen (ArtefactsScreen).

Если же игрок пережил все волны, то, в духе рогаликов, нам открывается меню с выбором одного из нескольких предметов. Подробнее о системе предметов на странице Artefacts Systems.

Здесь можно покинуть игру или продолжить. Однако не выбрав предмета - продолжить не получится.

Три бокса с описанием, иконкой и названием предмета - классика. При выборе предмета кнопка SELECT меняется на SELECTED и перекрашивается в зелёный. При выборе другого → прошлая возвращается в исходное состояние, а новая приобретает эффект выбора.



Интерфейс выбора предметов.



Выбранный предмет меняет надпись и подсвечиваемость.

```

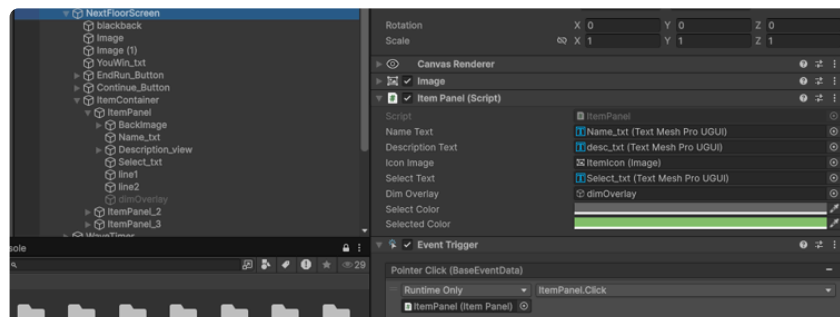
1 using System;
2 using TMPro;
3 using UnityEngine;
4 using UnityEngine.UI;
5
6 public class ItemPanel : MonoBehaviour
7 {
8     public Action<ItemPanel> OnClicked;
9
10     [SerializeField] private TMP_Text nameText;
11     [SerializeField] private TMP_Text descriptionText;
12     [SerializeField] private Image iconImage;

```

```

13 [SerializeField] private TMP_Text selectText;
14 [SerializeField] private GameObject dimOverlay;
15
16 [SerializeField] private Color selectColor;
17 [SerializeField] private Color selectedColor;
18
19 private ArtefactSO artefact;
20
21 // Создаём класс под панели. Храним ссылки на все элементы панели, а та
22 // текста. Создаем поле под ArtefactSO - ScriptableObject.
23
24 public void Initialize(ArtefactSO data)
25 {
26     gameObject.SetActive(true);
27     artefact = data;
28     nameText.text = data.name;
29     descriptionText.text = data.description;
30     iconImage.sprite = data.icon;
31
32     SetSelected(false);
33     dimOverlay.SetActive(false);
34 }
35
36 // Initialize(ArtefactSO) - инициализируем наш интерфейс данными из
37 // пришедшего пакета данных.
38
39 public void Click()
40 {
41     OnClicked?.Invoke(this);
42 }
43
44 // Click() - вызывает событие клика. На родительском объекте панели вис
45 // IventTrigger компонент, который обрабатывает клик и вызывает метод C
46 // К слову, событие будет обрабатывать отдельный класс, он будет и вызы
47 // SetSelected();
48
49 public void SetSelected(bool selected)
50 {
51     selectText.text = selected ? "SELECTED" : "SELECT";
52     selectText.color = selected ? selectedColor : selectColor;
53     dimOverlay.SetActive(!selected);
54 }
55
56 // SetSelected - эффект выбора панели. Меняет цвет и надпись, а
57 // также выключает затемнение.
58
59 public ArtefactSO GetArtefact() => artefact;
60
61 // GetArtefact() - возвращает сам SO артефакт. Понадобиться, когда игро
62 // выбрать панель и нажимать продолжить, чтобы применить артефакт.
63
64 public void Clear()
65 {
66     nameText.text = "";
67     descriptionText.text = "";
68     iconImage.sprite = null;
69     SetSelected(false);
70     selectText.text = "SELECT";
71     selectText.color = selectColor;
72     dimOverlay.SetActive(false);
73     gameObject.SetActive(false);
74 }
75 // Clear() - сбрасывает интерфейс и поле артефакта, чтобы повесить след
76 }

```



Иерархия и инспектор.

Создаём ScriptableObject класс, который будет конфигом для артефакта. А также сам эффект тоже делаем SO. Это позволит в артефакт пробрасывать SO настраиваемый, самостоятельный вариант эффекта.

```

1 using UnityEngine;
2
3 [CreateAssetMenu(menuName = "Artefacts/Artefact")]
4 public class ArtefactSO : ScriptableObject
5 {
6     public int IDA;
7     public string artefactName;
8     [TextArea] public string description;
9     public Sprite icon;
10
11     public ArtefactEffectSO effectArtefact;
12 }
13
14 public abstract class ArtefactEffectSO : ScriptableObject
15 {
16     public abstract void Equip();
17     public abstract void Dequip();
18 }

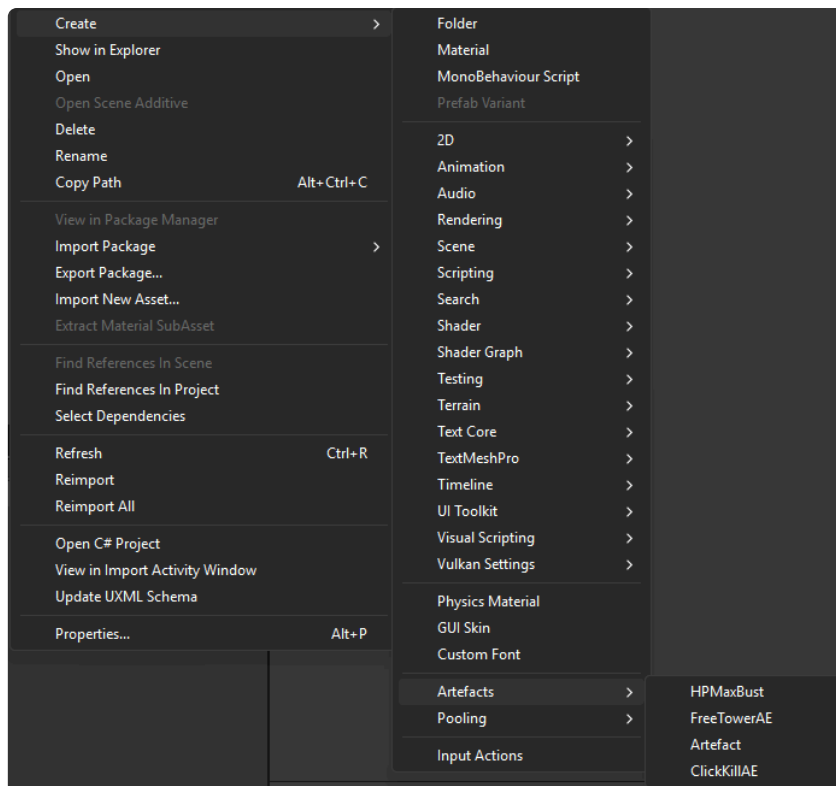
```

Пример предмета:

```

1 using UnityEngine;
2
3 [CreateAssetMenu(menuName = "Artefacts/ClickKillAE")]
4 public class ClickKillAE : ArtefactEffectSO
5 {
6     public override void Dequip()
7     {
8         Debug.LogError("Artefact 3 Dequip");
9     }
10
11     public override void Equip()
12     {
13         Debug.LogError("Artefact 3 Equip");
14     }
15 }

```



Создание SO эффекта.


```

8  /*
9  ContinueButton() - вызывается нажатием кнопки на интерфейсе.
10 Передаёт в ArtefactManager выбранный предмет и вызывает продолжение игр
11 GameFlowManager.
12
13 Интересный момент - проверки на то, что предмет выбран - нет. Хотя и
14 обойтись флагом, решением стало просто сделать кнопку некликабельной. А
15 только выбирается хотя бы один предмет (клик по панели вызывает событие
16 событие вызывает ClickedPanel(передавая себя же)) - то кнопка становится
17 А вернуться к состоянию отсутствия выбора нельзя.
18 */
19 public void EndRunButton() =>
20     GameFlowManager.Instance.Defeat();
21
22 public void LoadNewItems()
23 {
24     ClearPanels();
25     var items = ArtefactManager.Instance.GetRandomArtefacts(itemAmo
26
27     int count = Mathf.Min(items.Count, itemPanels.Count);
28     if (count < 1) {
29         Debug.LogError("Not a single item was received from the Artef
30     }
31     for (int i = 0; i < count; i++)
32     {
33         itemPanels[i].Initialize(items[i]);
34     }
35 }
36
37 /*
38 LoadNewItems() - очищаем панели, получаем список случайных предметов
39 менеджера, проверяем на меньшее значение. И инициализируем присутствующ
40 В случае если предметов будет меньше панелей, то панели не инициализир
41 не включаться.
42 */
43
44 private void ClearPanels()
45 {
46     foreach (var item in itemPanels)
47     {
48         item.Clear();
49     }
50     selectedPanel = null;
51     selectedArtefactSO = null;
52     continueButton.interactable = false;
53 }
54
55 // ClearPanels() - вызываем у каждой панели очистку интерфейса,
56 // устанавливаем все наши поля на null/false.
57
58 // Метод не очищает панели как объекты, поэтому отписки пока не требуют
59 // Панели по стандарту будет три. Для игры этого может быть достаточно
60 // Однако, логику можно будет слегка изменить, чтобы допустить возможно
61 // динамически, по ходу забега. В таком случае отписки следует добавить
62
63
64 private void ClickedPanel(ItemPanel panel)
65 {
66     if (panel == selectedPanel) { return; }
67     selectedPanel = panel;
68     continueButton.interactable = true;
69
70     foreach (var item in itemPanels)
71     {
72         if (selectedPanel == item)
73         {
74             selectedPanel.SetSelected(true);
75             selectedArtefactSO = selectedPanel.GetArtefact();
76         }
77         else item.SetSelected(false);
78     }
79 }
80 /*
81 ClickedPanel(ItemPanel panel) - реагирует на событие и получает панел
82 по которой кликнули. Если панель и так выбрана - игнорируем. Если нет -
83 панель, ищем среди листа панелей выбранную - Активируем и получаем её п
84 */
85 }

```

Сейчас NextFloorScreen работает с панелями по ссылкам из инспектора. А количество предметов жестко выставлено в три. Для данного проекта этого достаточно и он не предполагает наличие большего количества. Однако, расширить систему можно очень легко, немного изменив NextFloorScreen, создавая нужное количество панелей или просто переиспользуя большее количество, чем 3. (Даже сейчас можно указать в лист больше панелей, создав их на сцене, просто использоваться будет только 3. Можно будет в таком случае указать рандомный дроп 2-5 предметов на выбор, но для меня это не нужно).

Информация про ArtefactManager будет находиться на странице Artefacts System.

ESC Screen.

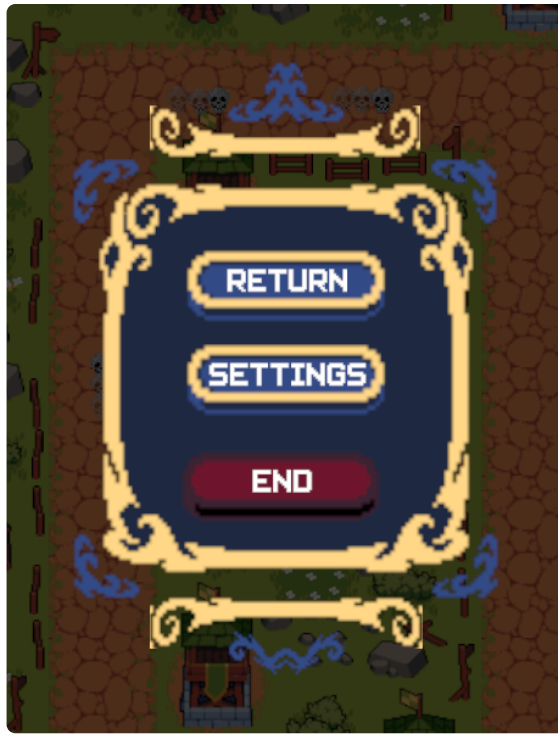
- В подавляющем большинстве игр есть меню-паузы. Его роль остановить игру, а также предоставить некий функционал: настройки, информацию о частях игры, выход и прочее. При нажатии клавиши ESC (или кнопки на интерфейсе) - у меня в игре вызывается ESC menu.



Esc-Menu.



Esc-Menu: Settings Button.



Повторная проверка кнопки выхода.

Return - возвращает нас в игру, и включает прежнюю скорость. Settings - вызывает меню настроек. End - сначала загорается красным, и лишь при повторном нажатии вызывает экран завершения игры, со всеми статистическими данными.

```

1  using TMPro;
2  using UnityEngine;
3  using UnityEngine.EventSystems;
4  using UnityEngine.InputSystem;
5  using UnityEngine.UI;
6
7  public class ESCScreen : MonoBehaviour
8  {
9      [SerializeField] private GameObject root;
10     [SerializeField] private GameObject endrunButtonObj;
11
12     [SerializeField] private Sprite red_endrunButtonSprite;
13     [SerializeField] private Sprite red_endrunButtonHighlighterSprite;
14     [SerializeField] private Sprite red_endrunButtonPressedSprite;
15
16     private bool settingsOn = false;
17
18     private Button endrunButton;
19     private TMP_Text endrunText;
20     private Image endrunImage;
21     private Sprite defaultSprite;
22     private SpriteState defaultState;
23     private SpriteState redState;
24
25
26     private bool endrunPress = false;
27
28     private void Start()
29     {
30         endrunText = endrunButtonObj.GetComponentInChildren<TMP_Text>();
31         endrunImage = endrunButtonObj.GetComponent<Image>();
32         defaultState = (endrunButtonObj.GetComponent<Button>()).sprite
33         defaultSprite = endrunImage.sprite;
34         endrunButton = endrunButtonObj.GetComponent<Button>();
35
36         redState = defaultState;
37         redState.highlightedSprite = red_endrunButtonHighlighterSprite;
38         redState.pressedSprite = red_endrunButtonPressedSprite;
39
40         root.SetActive(false);
41     }

```

```

42
43
44     public void Hide()
45     {
46         root.SetActive(false);
47         if (endrunPress)
48         {
49             ResetEndButton();
50         }
51         if (settingsOn)
52         {
53             SettingsMenuController.Instance.Hide();
54             settingsOn = false;
55         }
56
57         GameFlowManager.Instance.ReturnPlaying();
58     }
59
60     // Hide() - скрывает меню и вызывает ReturnPlaying(), который вызовет
61     // восстановление скорости игры.
62
63     public void Show()
64     {
65         if (GameFlowManager.Instance.State != GameState.Playing) return
66         root.SetActive(true);
67         GameFlowManager.Instance.EscMenu();
68     }
69
70     // Show() - показывает меню, вызывая состояние EscMenu, что ставит игр
71     // логическую паузу. Чтобы меню нельзя было вызвать повторно или где-
72     // вне игры, проверяем состояние на Playing.
73
74     public void EscKeyPress(InputAction.CallbackContext ctx)
75     {
76         if (!ctx.performed) return;
77         if (root.activeSelf)
78         {
79             Hide();
80             if (settingsOn)
81                 SettingsMenuController.Instance.Hide();
82         }
83         else
84         {
85             Show();
86         }
87     }
88     // EscKeyPress - через карту событий обрабатываем нажатие ESC клавиши.
89     // Если меню закрыто - открываем, если открыто - закрываем. Помимо это
90     // Show() также находится на кнопке в верхней панели слева.
91
92     public void ReturnButton() => Hide();
93
94     // ReturnButton - скрываем панель. SettingsButton - вызываем меню наст
95     // EndRunButton - завершаем забег.
96
97     // При закрытии панели важно также закрыть меню настроек, и вернуть кн
98     // выхода прежнее состояние, если вдруг игрок передумал выходить
99     // (подтверждать выход).
100
101     public void SettingsButton()
102     {
103         if (!SettingsMenuController.Instance.isActiveAndEnabled)
104         {
105             SettingsMenuController.Instance.Show();
106             settingsOn = true;
107         }
108         else if (SettingsMenuController.Instance.isActiveAndEnabled)
109         {
110             SettingsMenuController.Instance.Hide();
111             settingsOn = false;
112         }
113     }
114
115     public void EndRunButton()
116     {
117         if (endrunPress) { EndButton(); return; }
118         endrunPress = true;
119         SetRedButton();
120         endrunText.text = "END";
121     }
122
123     public void EndButton()

```

```
124     {
125         Hide();
126         GameFlowManager.Instance.Defeat();
127     }
128
129     public void ResetEndButton()
130     {
131
132         endrunText.text = "ENDRUN";
133         endrunPress = false;
134         SetDefaultButton();
135     }
136
137     private void SetRedButton()
138     {
139         endrunImage.sprite = red_endrunButtonSprite;
140         endrunButton.spriteState = redState;
141     }
142
143     private void SetDefaultButton()
144     {
145         endrunImage.sprite = defaultSprite;
146         endrunButton.spriteState = defaultState;
147     }
148 }
```